Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Programlama dilleri
  - Programlama dilleri biçimsel dillerdir (formal languages): her ifadenin tek bir anlamı vardır. Doğal dillerde (Türkçe, İngilizce) ise bir ifade birden fazla anlama gelebilir.
  - Programlama dili aileleri arasında yapısal programlama dilleri (structured programming languages) ve nesneye yönelik programlama dilleri (object-oriented programming languages) vardır. Gömülü sistemler gibi özel ortamlar için de diller bulunur.
  - Seviyeler: alt seviye (makine kodu, assembly), orta seviye (C), yüksek seviye (Java, SQL...). Donanımdan ve işlemci komut setinden uzaklaştıkça dil konuşma diline yaklaşır.
  - Alt seviye diller ilk bakışta zor anlaşılır ve uzun kod gerektirir, ama sistemi en verimli şekilde kullanmayı sağlar. Yüksek seviye diller kolay anlaşılır ve kısa komutlarla çok iş yapar.
  - C genelde orta seviye bir dil olarak geçer (bazı kaynaklarda yüksek seviye). Tüm sistem kaynaklarını yönetebilir ve alt seviye dillere göre daha anlaşılırdır. Python ise C üzerine kurulmuştur ve çok daha az kodla iş yapar.
- Derlenen ve yorumlanan diller
  - Derlenen diller (compiled languages): kaynak kod (source file) derleyici (compiler) ile nesne koduna (object code) çevrilir. Birden fazla kaynak dosya varsa birden fazla nesne kodu oluşur. Bağlayıcı (linker) bunları tek bir çalıştırılabilir koda (executable) birleştirir. C bu mantıkla çalışır.
  - Derlenen diller genellikle daha hızlı çalışan programlar üretir.
  - Yorumlanan diller (interpreted languages): yorumlayıcı (interpreter) kaynak kodu satır satır çevirir ve hemen çalıştırır. Tüm kod bir kerede makine diline çevrilmediği için daha yavaştırlar. Örnek: MATLAB, Python, JavaScript.
  - Hem derlenen hem yorumlanan diller de vardır. Örnek: Java. Java kodu önce bytecode'a derlenir, sonra Java Virtual Machine (JVM) tarafından yorumlanır. Bu sayede platformdan bağımsızdır: Windows'ta yazılan kod Linux'ta da yeniden derlenmeden çalışır.
  - C'de belirli bir işlemci mimarisi için derlenen çalıştırılabilir kod yalnızca o mimaride çalışır. Başka mimaride çalıştırmak için yeniden derlemek gerekir.
- C programlama dili
  - 1972'de Unix işletim sistemini yazmak için geliştirilmiştir. Öncülü B dilidir.
  - Bir işletim sistemini yazabilecek kadar güçlü olması, bellek yönetimi ve kaynaklara doğrudan erişim sağlamasından gelir.
  - Yaklaşık 50 yıldır popülerliğini korur. Java, C#, C++ gibi birçok dilin sözdizimi (syntax) C'ye benzer.
  - Çok güçlüdür, ama kendine özgü özelliklerine hâkim olunmazsa yanlış çalışan programlar yazmak kolaydır.
- C programının temel yapısı (ekrandan, 00:33:50)

```c
#include <stdio.h>
int main() {
    int sayi1, sayi2, sonuc;
```

```
    printf("Bir deger giriniz"); // Comment Line: Ilk sayiyi giriniz
    scanf("%d", &sayi1);
    // Comment Line: 2. sayiyi giriniz
    printf("Ikinci degeri giriniz");
    scanf("%d", &sayi2);
    sonuc = sayi1 + sayi2;
    printf("Sonuc = %d", sonuc);
    return 0;
}
```

  - `#include <stdio.h>`: giriş/çıkış fonksiyonlarını (`printf`, `scanf`) içeren kütüphane. Her programda eklenmelidir. Diğer kütüphaneler: `stdlib.h`, `string.h` vb.
  - `int main()`: her C programında bulunması gereken ana fonksiyon. Fonksiyonların gövdesi süslü parantezle açılıp kapanır.
  - `int` dönüş tipi, main'in bir tam sayı değer döndüreceğini belirtir. `return 0;` programın sorunsuz bittiğini ve kontrolün işletim sistemine geçtiğini bildirir.
  - C'de değişkenler kullanılmadan önce tanımlanmalıdır (akış diyagramından farklı olarak).
  - `scanf("%d", &sayi1);`: & işareti değişkenin adresini verir. `scanf` ile okumada & kullanılır (ileride görülecek bir istisnası vardır). `printf`'te adres işareti kullanılmaz.
  - `printf("Sonuc = %d", sonuc);`: `%d`, ilk görüldüğü yere `sonuc` değişkeninin değerini yazar.
  - Her ifadenin (statement) sonuna noktalı virgül konur.
  - Açıklama satırları: `//` tek satırlık, `/* ... */` çok satırlık açıklama (comment) yazar. Açıklamalar derlenmez. Kodu başkasının ya da daha sonra kendinizin anlaması için eklenir.

- Operatörler (ekrandan, 00:41:40)
  - Aritmetik: `+`, `-`, `*`, `/`, `%` (mod).
  - Atama: `=` (sağdan sola atar).
  - İlişkisel (relational): `<`, `>`, `<=`, `>=`, `==` (eşit mi), `!=` (eşit değil mi).
  - Mantıksal (logical): `&&` (AND, ve), `||` (OR, veya). Genellikle if ve while koşullarında kullanılır.
- Değişken tipleri
  - Tam sayı: `int`. Daha küçük alan için `short int`, daha büyük sayılar için `long int`.
  - Kesirli sayı: `float`. Daha hassas: `double`, `long double`.
  - Karakter: `char`. C'de her zaman 1 byte kaplar.
  - Birden fazla karakter için ayrı bir tip yoktur, karakter dizisi kullanılır: `char kelime[10];`.
  - `unsigned` anahtar kelimesi işaretsiz tip tanımlar: `unsigned int`, `unsigned char`. `unsigned char` 0-255, `char` -128 ile 127 aralığını saklar.
  - `char` küçük sayısal değerleri saklamak için de kullanılabilir, böylece daha az bellek harcanır. `int` genelde 4 byte, `short int` 2 byte kaplar. Bu boyutlar mimariye göre değişebilir, `char` ise her zaman 1 byte'tır.
- Format belirteçleri
  - `%d` tam sayı (decimal), `%c` karakter, `%f` float, `%lf` double, `%s` karakter dizisi (string).
- Geliştirme ortamları
  - Derleyici (compiler) ile geliştirme ortamı farklı kavramlardır. GCC bir derleyicidir. Notepad++ bir editördür.
  - IDE (Integrated Development Environment) editör, derleyici ve hata ayıklayıcıyı (debugger) bir arada sunar. Örnekler: Dev-C++, Code::Blocks, CodeLite, CLion, VS Code, Xcode.
  - Hata ayıklama (debug), koddaki hataların bulunup ayıklanmasıdır.
  - Kod bir editörde yazılıp terminalden derlenerek de çalıştırılabilir. Online derleme ortamları da vardır.
  - ANSI (American National Standards Institute) standartlarına uygun yazılan kod tüm ortamlarda sorunsuz çalışır. Belirli bir işletim sistemine özgü geliştirme diğer ortam-

larda sorun çıkarabilir.

- Koşullu dallanma (if-else) (ekrandan, 01:13:40)
  - Yapı: `if (x) statement1; else statement2;`
  - Parantez içindeki değer 0 dışında herhangi bir değerse true kolu, 0 ise else kolu çalışır. `if (-1)` true koluna, `if (0)` else koluna gider.
  - Karşılaştırma ifadeleri de değer döndürür: doğruysa 1, yanlışsa 0. Örneğin `x = 4` iken `if (x == 4)` 1 döndürür ve true kolu çalışır.
  - Süslü parantez kullanılmazsa koşula yalnızca hemen sonraki tek ifade bağlıdır:

    ```
    if (x)
        statement1;
    else
        statement2;
    statement3; /* Always executed */
    ```

    `statement3`'ün else koluna ait olması için süslü parantez gerekir:

    ```
    if (x) {
        statement1;
    } else {
        statement2;
        statement3;
    }
    ```

  - Hoca, tek ifade olsa bile her zaman süslü parantez kullanılmasını tavsiye eder.
- İç içe if (nested if) örneği: üç sayının en küçüğü

```
int min(int a, int b, int c) {
    if (a < b)
        if (a < c)
            return a;
        else
            return c;
    else if (b < c)
        return b;
    else
        return c;
}
```

  - Her `else`, kendisinden önceki en yakın eşleşmemiş `if` ile eşleşir.
  - `return`, fonksiyondan çağırana değer döndürür. Fonksiyonlar ve modüler programlama ileride işlenecektir. Bu dönem tüm işler `main` içinde yapılacak, sonuçlar `printf` ile yazdırılacaktır.
- Döngüler (ekrandan, 01:23:30)
  - `while`: önce koşul kontrol edilir, sağlanıyorsa gövde çalışır.

    ```
    while (x < y) {
        x++;
        y--;
    }
    ```

  - `do...while`: önce gövde bir kez çalışır, sonra koşul kontrol edilir. `while (...)` satırının sonuna noktalı virgül konur (istisna).

    ```
    do {
        x++;
        y--;
    } while (x < y);
    ```

  - `for`: üç ifadesi vardır. `for (expr1; expr2; expr3) { statement; }`

```c
for (i = 0; i < n; i++) {
    toplam = toplam + dizi[i];
}
```

  * Çalışma sırası: önce expr1 (ilk değer atama), sonra expr2 (koşul), koşul doğruysa gövde (statement), sonra expr3 (artırma). Ardından tekrar expr2 kontrol edilir.
  * expr1'de virgülle birden fazla ilk değer atanabilir: `for (i = 0, j = 0; ...)`.
  * `i++` yerine `i--` yazılırsa (i = 0, n pozitif) koşul hiç bozulmaz ve sonsuz döngü oluşur.

- C'de dizi indisleri 0'dan başlar: `dizi[0] ... dizi[n-1]`. Akış diyagramında 1'den N'e kadar kullanılan döngüler koda geçerken 0'dan n−1'e uyarlanmalıdır. Akış diyagramında da 0'dan başlatmak yanlış değildir.

Hocanın Özellikle Vurguladığı Kısımlar

- C programında unutulmaması gerekenler
  - `stdio.h` kütüphanesini eklemek, `main` fonksiyonunu açıp kapatmak, `return 0;` ile bitirmek, okuma için `scanf`, yazma için `printf` kullanmak.
- Süslü parantez
  - If ve else kollarında, tek ifade olsa bile süslü parantez kullanmak hataları önler.
- if içindeki değer
  - 0 dışındaki her değer doğru, 0 yanlış kabul edilir.
- `scanf`'te &
  - Değişkene okuma yapılırken adres işareti kullanılmalıdır.
- ANSI standardı
  - Kodun her ortamda çalışması için standart C yazılmalıdır.
- Akış diyagramlarını koda çevirin
  - Derste çözülen 13 civarı akış diyagramının C koduna dönüştürülüp çalıştırılması iyi bir alıştırmadır.
- Algoritma tasarımı daha önemli
  - Bu aşamada algoritma tasarlamak kod yazmaktan daha önemlidir. Kodlama birlikte, uygulama derslerinde pekiştirilecektir.

Kısa Tekrar Notları

- Programlama dilleri biçimseldir: her ifade tek anlamlıdır.
- Compiled: kaynak kod → derleyici → object code → linker → executable (C). Interpreted: satır satır yorumlanır (MATLAB, Python). Java: bytecode + JVM.
- C: 1972, Unix için. Orta seviye dil.
- Program iskeleti: `#include <stdio.h>`, `int main() { ... return 0; }`.
- `scanf("%d", &x);` (adres), `printf("%d", x);` (adres yok).
- Yorum: `//` ve `/* ... */`.
- Operatörler: `+ - * / %`, `=`, `< > <= >= == !=`, `&& ||`.
- Tipler: `int`, `short int`, `long int`, `float`, `double`, `long double`, `char` (1 byte), `unsigned`.
- Format: `%d %c %f %lf %s`.
- IDE = editör + derleyici + debugger.
- `if (0)` → else. Sıfır dışı → true. Süslü parantez kullanın.
- while: önce kontrol. do-while: önce iş (sonunda `;`). for: expr1 → expr2 → gövde → expr3.
- C dizileri 0'dan başlar.

Detaylı Açıklamalar

Bu derste algoritma tasarımından C programlama diline geçiş yapılır. Önce programlama dillerinin genel sınıflandırması ele alınır. Programlama dilleri biçimsel dillerdir, yani doğal dillerdeki gibi bir ifadenin birden fazla anlamı yoktur ve bilgisayar her ifadeyi tek bir biçimde yorumlar. Diller donanıma yakınlıklarına göre alt, orta ve yüksek seviye olarak ayrılır. Donanıma yaklaştıkça diller daha verimli ama daha zor, uzaklaştıkça daha kolay ama daha az denetimli hâle gelir. C bu ikisi arasında, sistem kaynaklarına tam erişim sağlayan ve görece anlaşılır bir orta seviye dildir.

Diller ayrıca çalıştırılma biçimlerine göre ayrılır. Derlenen dillerde yazılan kaynak kod derleyici tarafından nesne koduna, ardından bağlayıcı tarafından tek bir çalıştırılabilir dosyaya çevrilir. C bu yaklaşımı kullanır ve derlenen programlar genellikle hızlıdır, ancak her işlemci mimarisi için ayrı derlenmeleri gerekir. Yorumlanan dillerde ise yorumlayıcı kodu satır satır çevirerek çalıştırır. Bu yöntem daha yavaştır. Java ise kodu önce bytecode'a derleyip ardından JVM üzerinde yorumlayarak platformdan bağımsız çalışmayı sağlar.

C dili 1972'de Unix işletim sistemini yazmak için geliştirilmiştir. Bir işletim sistemini yazabilecek güçte olması, belleğe ve donanım kaynaklarına doğrudan erişim sağlamasındandır. Bu güç, dikkatli kullanılmadığında hatalı programlara da yol açabilir. Hoca, C'nin yaklaşık 50 yıldır kullanıldığını ve birçok modern dilin sözdizimini etkilediğini belirtir.

Temel bir C programı tahtada bir örnekle anlatılır. Program `stdio.h` kütüphanesinin eklenmesiyle başlar, çünkü ekrana yazma ve klavyeden okuma fonksiyonları bu kütüphanededir. Her C programında bir `main` fonksiyonu bulunur ve bu fonksiyon `return 0;` ile işletim sistemine programın başarıyla bittiğini bildirir. Akış diyagramlarından farklı olarak C'de değişkenler kullanılmadan önce tipleriyle tanımlanmalıdır. Kullanıcıdan değer okumak için `scanf` kullanılır. Biçim belirteci (`%d`) ve değişkenin adresi (`&sayi1`) verilir. Sonuç `printf` ile yazdırılırken biçim belirtecinin yerine değişkenin değeri konur. Her ifade noktalı virgülle biter. Açıklama satırları `//` ve `/* */` ile yazılır ve derlenmez.

Ardından C'deki operatörler ve değişken tipleri özetlenir. Aritmetik, atama, ilişkisel ve mantıksal operatörlerin C karşılıkları verilir. Özellikle karşılaştırma için `==`, mantıksal ve için `&&`, veya için `||` kullanıldığı vurgulanır. Tam sayılar için `int` ve türevleri, kesirli sayılar için `float` ve `double`, karakterler için `char` kullanılır. `unsigned` anahtar kelimesiyle işaretsiz tipler tanımlanır ve aynı bellek alanında daha büyük pozitif sayılar saklanabilir. `char` her zaman 1 byte olduğu için küçük sayıları az bellekle saklamak için de kullanılabilir.

Derste ayrıca derleyici ile geliştirme ortamı arasındaki fark açıklanır. Derleyici yalnızca kaynak kodu makine diline çevirir. IDE ise kod yazmayı kolaylaştıran bir editör, derleyici ve hata ayıklayıcıyı bir araya getirir. Hoca, standart dışı ortamlar yerine ANSI standartlarına uyan ortamların tercih edilmesini önerir.

Son bölümde kontrol akışı yapılarının C'deki karşılıkları gösterilir. If yapısında koşul ifadesinin değeri 0 değilse true kolu, 0 ise else kolu çalışır. Karşılaştırma ifadeleri doğruysa 1, yanlışsa 0 değerini üretir. Süslü parantez kullanılmadığında yalnızca hemen sonraki tek ifadenin koşula bağlı olduğu bir örnekle gösterilir. Bu yüzden her zaman süslü parantez kullanılması tavsiye edilir. Üç sayının en küçüğünü bulan iç içe if örneğiyle her `else`'in hangi `if` ile eşleştiği açıklanır. Döngülerden while önce koşulu kontrol eder, do-while ise gövdeyi en az bir kez çalıştırdıktan sonra koşulu kontrol eder. For döngüsünde ilk değer atama, koşul ve artırma ifadeleri tek satırda yazılır. Çalışma sırası ilk değer, koşul, gövde ve artırma şeklindedir. Artırma yerine yanlışlıkla azaltma yazılması sonsuz döngüye yol açabilir. C'de dizi indisleri 0'dan başladığı için akış diyagramlarındaki 1'den N'e döngüler koda geçerken 0'dan $n-1$'e uyarlanmalıdır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.